Graph-Based Context Retrieval for Retrieval-Augmented Generation:

Personalized PageRank as the Main Retrieval Model on the Wiki-Vote Network

Net RAG Project

Abstract. Retrieval-augmented generation (RAG) systems improve language-model reliability by grounding generation in external evidence, but conventional retrieval can miss structural context when relevance is expressed through indirect relationships rather than lexical similarity. This paper presents a compact GraphRAG-style retrieval module that formulates context retrieval as link prediction on the Wikipedia Vote Network. The main retrieval model is approximate Personalized PageRank, evaluated against local graph heuristics and additional horizontal baselines. The system parses raw SNAP Wiki-Vote edges, performs source-node-stratified train/test splits, constructs controlled positive-negative candidate sets, and measures ranking quality with Precision@K, HitRate@K, and MRR. On the original full-dataset setting with 82,822 training edges and 20,867 hidden test edges, Personalized PageRank obtains Precision@10 of 0.239297, HitRate@10 of 0.791845, and MRR of 0.551049. A broader horizontal evaluation over five seeds, three negative-sampling strategies, five K cutoffs, and nine graph scorers further shows that Personalized PageRank is the strongest model under random and hard two-hop negative sampling, while degree-matched negatives expose the competitiveness of normalized local-overlap methods. The study demonstrates that graph-based retrieval can provide a reproducible, inspectable retrieval layer for GraphRAG pipelines before adding a downstream language generator.

Keywords: GraphRAG; retrieval-augmented generation; link prediction; Personalized PageRank; Wiki-Vote.

1 Introduction

Large language models can generate fluent answers while still failing when the required knowledge is absent, private, or stale. Retrieval-augmented generation addresses this limitation by retrieving external evidence before generation, allowing model outputs to be grounded in non-parametric memory [10]. However, a retrieval component based only on direct lexical or vector similarity can miss relevant context that is expressed through relationships. Recent GraphRAG work argues that graph indexes can improve retrieval and summarization when questions require global or relational context [2].

This project studies the retrieval part of that idea in isolation. Instead of constructing a full question-answering application, it builds a graph retrieval module over the Wikipedia Vote Network. A directed edge from user u to user v denotes that u voted for v in a Wikipedia administrator election. If an unobserved edge from u to v can be predicted from the training graph, then v is treated as structurally relevant context for u.

The contribution is practical and reproducible. The repository implements raw data parsing, deterministic edge splitting, candidate generation, graph retrieval algorithms, ranking evaluation, single-user context retrieval, horizontal model comparison, and visualization. The central model studied in the paper is approximate Personalized PageRank, with other graph scorers serving as baselines or robustness checks. This paper summarizes the implemented method and reports the saved full-dataset results using the project artifacts.

2 Related Work

Retrieval has long been a central mechanism for grounding natural language systems. Classical sparse retrieval methods such as BM25 are rooted in probabilistic relevance modeling and remain strong baselines for document search [14]. Dense passage retrieval later showed that dual-encoder representations can retrieve semantically related passages even when exact lexical overlap is weak [6]. RAG combines this retrieval step with a parametric generator, allowing language models to condition generation on external non-parametric memory [10]. This design is especially relevant for domains where model parameters alone cannot guarantee factuality, freshness, or coverage.

GraphRAG extends retrieval beyond independent text chunks. Instead of treating candidate contexts as isolated documents, graph-based retrieval models organize entities, documents, and relations into a structured index. This enables systems to retrieve context through relational paths, community structure, or global summaries rather than only through vector similarity. Recent GraphRAG work demonstrates that graph indexes can support query-focused summarization over large collections by combining local evidence with global community-level context [2]. The present project follows the same motivation but narrows the research question to the retrieval module: can graph structure alone recover hidden relevant nodes?

The task is also connected to classical link prediction. Liben-Nowell and Kleinberg formalized link prediction as inferring missing or future social interactions from a network snapshot and showed that graph proximity features can be predictive [11]. Common Neighbors and Jaccard are local neighborhood-overlap heuristics; Adamic-Adar refines overlap by assigning more value to

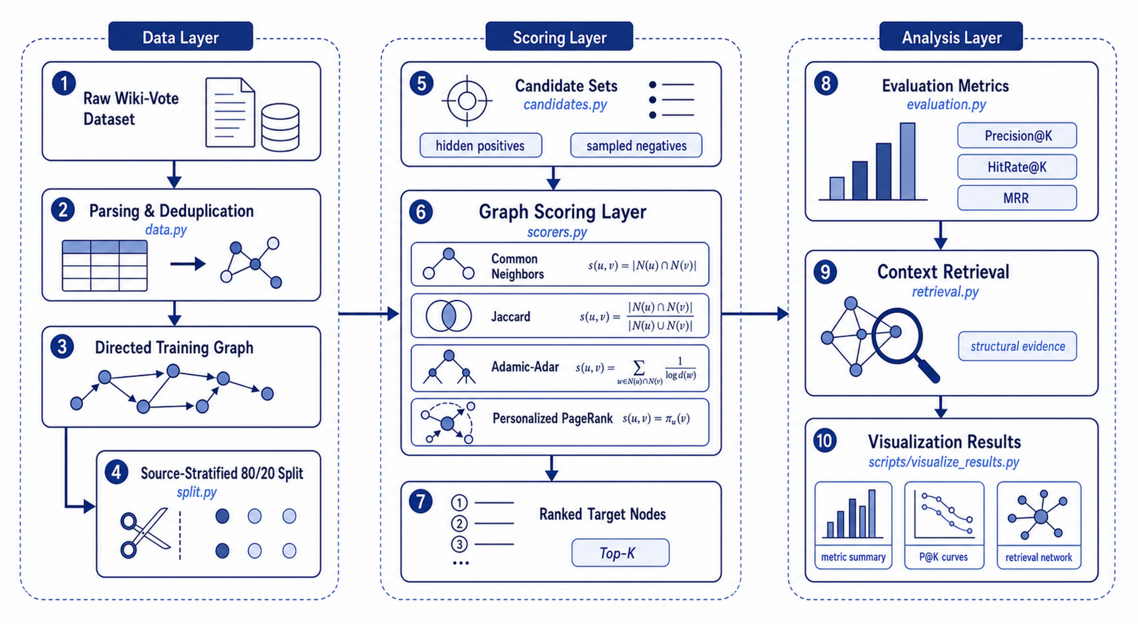


Figure 1: Implemented Net RAG workflow. The repository separates the system into a data layer, a graph scoring layer, and an analysis layer, allowing retrieval quality to be measured before integration with a downstream language generator.

rare shared neighbors [1]. PageRank and Personalized PageRank provide random-walk views of node importance and source-conditioned relevance [5, 12]. These methods are attractive for a compact GraphRAG retrieval module because they are interpretable, deterministic under fixed seeds, and do not require task-specific neural training.

A parallel line of graph representation learning learns continuous node embeddings from random walks or biased neighborhood sampling. DeepWalk generalizes language-model-style sequence learning to graph walks [13], while node2vec introduces biased random walks to balance local and structural neighborhoods [3]. Such embedding approaches can be powerful for downstream prediction, but they introduce additional training objectives and hyperparameters. This project instead emphasizes transparent graph scorers, which makes the retrieval behavior easier to audit in a small course-scale GraphRAG pipeline.

The experimental dataset comes from SNAP's Wikipedia Vote Network, whose source material is associated with work on signed social networks and link prediction in online communities [7–9]. The implementation uses NetworkX for graph construction and traversal [4].

3 Methodology

3.1 Problem Formulation

Figure 1 summarizes the implemented workflow. Let $G = (V, E)$ be the directed Wiki-Vote graph. The project hides a subset of edges E_{test} and builds a training graph $G_{\text{train}} = (V, E_{\text{train}})$. For a source node s, the retrieval module ranks candidate targets t that are not already visible as training outgoing edges from s. Hidden test edges are positive candidates, and randomly sampled non-existent edges are negative candidates.

The output is a ranked list:

$$R_s = \text{rank}_{t \in C_s}\, f(s,t),$$

where C_s is the candidate set for source s and f is a graph scoring function.

3.2 Data Preparation

The raw file `Wiki-Vote.txt` is parsed directly. Comment lines are skipped, edge rows are converted into integer node pairs, and duplicate edges are removed. Because the dataset does not include timestamps for each edge in the local task file, the project uses a deterministic random split. For each source node with at least two outgoing edges, approximately 20% of its targets are sampled into the test set, while at least one outgoing edge is kept for training. Sources with one outgoing edge remain in the training graph.

SNAP reports 7,115 nodes and 103,689 edges for the Wiki-Vote network [7]. The repository's default split with seed 42 produces 82,822 training edges and 20,867 testing edges from 103,689 unique edges.

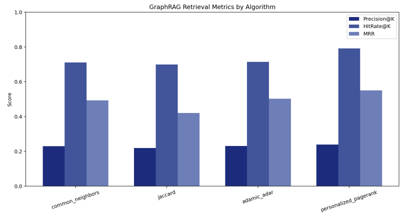
3.3 Graph Scoring Functions

The project uses approximate Personalized PageRank as the main GraphRAG retrieval model and evaluates local graph scorers as interpretable baselines. Common Neighbors counts neighbor overlap in an undirected projection:

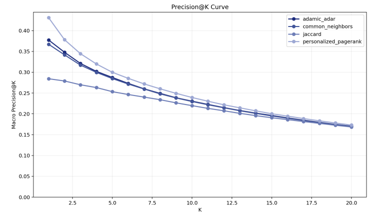
$$f_{\text{CN}}(s,t) = |\Gamma(s) \cap \Gamma(t)|.$$

The Jaccard coefficient normalizes overlap by the union size:

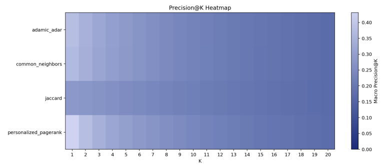
$$f_{\text{Jaccard}}(s,t) = \frac{|\Gamma(s) \cap \Gamma(t)|}{|\Gamma(s) \cup \Gamma(t)|}.$$



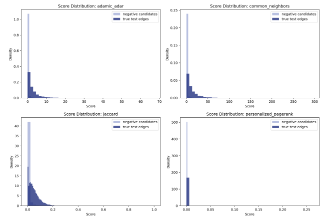
(a) Metric summary across graph retrieval algorithms.



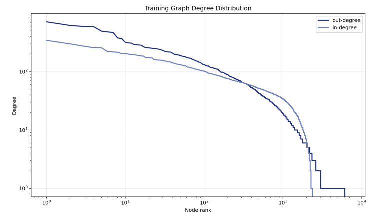
(b) Precision@K curve for cutoff sensitivity.



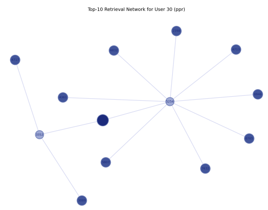
(c) Precision@K heatmap by scorer and cutoff.



(d) Positive-negative score distribution.

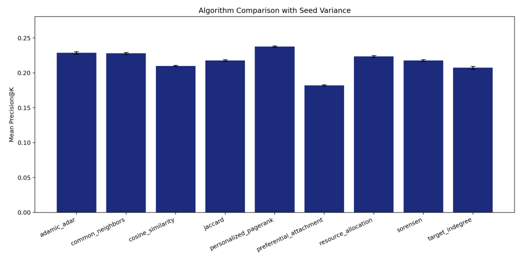


(e) Training graph degree distribution.

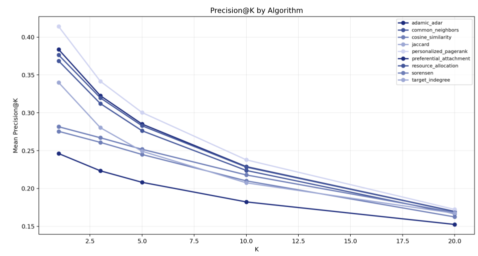


(f) Personalized PageRank retrieval network for user 30.

Figure 2: Consolidated diagnostic visualizations for the original retrieval evaluation. Panels (a)–(c) summarize algorithm-level ranking quality, panel (d) shows score separation between positive and negative candidates, and panels (e)–(f) connect retrieval behavior to graph structure and an example context set.



(a) Mean Precision@10 with seed-level variance across horizontal algorithms.



(b) Precision@K curves for the horizontal algorithm set.

Figure 3: Horizontal evaluation quality plots. Panel (a) compares average algorithm performance under the default random-negative protocol, and panel (b) shows how ranking quality changes across retrieval cutoffs.

Adamic-Adar discounts high-degree common neighbors:

$$f_{\mathrm{AA}}(s,t) = \sum_{z \in \Gamma(s) \cap \Gamma(t)} \frac{1}{\log |\Gamma(z)|}.$$

These three scorers use the undirected training projection to reduce sparsity.

Personalized PageRank is implemented as an approximate random-walk scorer on the directed training graph. For each source node, the implementation simulates source-centered walks with restart probability 0.15, 400 walks, and walk length 20. Candidate targets are scored by normalized visit frequency. This produces a practical approximation to source-conditioned graph relevance and serves as the paper's main retrieval model.

3.4 Candidate Sampling and Evaluation

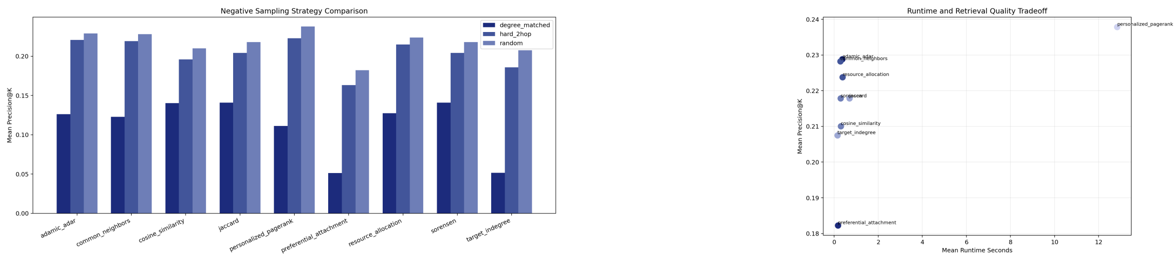
For every source node with hidden positives, the default candidate set includes its true hidden targets and up to 100 randomly sampled negative targets that are not edges in the full graph. The random sampler is deterministic under the configured seed. The horizontal evaluation extends this protocol with degree-matched negatives and hard two-hop negatives to test whether a scorer remains strong when popularity and local structural similarity are controlled.

The ranking is evaluated with Macro Precision@K, HitRate@K, and mean reciprocal rank (MRR). Precision@K measures the average fraction of top-K candidates that are true hidden edges. HitRate@K records whether at least one hidden edge appears in the top-K list. MRR rewards algorithms that place the first positive target early in the ranking.

4 Experiments: Personalized PageRank as the Main Model

The experiments are organized around approximate Personalized PageRank as the main GraphRAG retrieval model. The initial full-dataset experiment uses the repository defaults: seed 42, test ratio 0.2, $K = 10$, and 100 random negative samples per evaluated source. The full pipeline is executed from the repository root in four stages:

1. prepare the train/test edge split from `Wiki-Vote.txt`;



(a) Algorithm sensitivity to random, degree-matched, and hard two-hop negatives.

(b) Runtime-quality trade-off at the primary evaluation cutoff.

Figure 4: Horizontal evaluation robustness and efficiency plots. Panel (a) shows that the candidate-sampling protocol changes the ranking of graph scorers, while panel (b) exposes the additional runtime cost of Personalized PageRank relative to local heuristics.

2. evaluate graph scorers over sampled candidate sets;
3. retrieve the top related nodes for a selected source user;
4. generate summary plots from saved metrics and ranking files.

The experiments were conducted on a Python 3.12 workstation equipped with an NVIDIA GeForce RTX 4080 GPU. The software stack uses NetworkX for graph construction, neighborhood traversal, and random-walk-based retrieval. NumPy and pandas support deterministic candidate processing and metric aggregation, while matplotlib generates the visual analysis figures. The experimental artifacts are written as CSV, JSON, and PNG files so that each stage can be inspected and reproduced independently.

The split protocol is source-node stratified so that evaluated sources retain visible outgoing graph context in the training graph. This design is important for retrieval: if all outgoing edges of a low-degree source were hidden, neighborhood-based methods would have no local evidence from which to rank targets. Candidate construction combines all hidden positives for each evaluated source with a fixed-size negative set sampled from non-edges. This creates a controlled ranking problem while avoiding the infeasible cost of scoring all possible non-edges.

To strengthen the evaluation beyond a single baseline run, the horizontal evaluation compares nine graph scorers over five random seeds, five retrieval cutoffs ($K = 1, 3, 5, 10, 20$), and three negative-sampling strategies. The scorer set includes the four original models–Common Neighbors, Jaccard, Adamic-Adar, and Personalized PageRank–plus Resource Allocation, Preferential Attachment, Cosine Similarity, Sorensen, and Target In-Degree. The three candidate protocols test complementary forms of difficulty: random negatives measure standard link-prediction ranking, degree-matched negatives control target popularity, and hard two-hop negatives emphasize structurally plausible false targets. The horizontal evaluation therefore measures not only whether Personalized PageRank is strong in the original setting, but also whether the main model remains competitive under harder candidate construction.

Table 1: Original full-dataset retrieval performance with $K = 10$ and 100 random negative candidates per source. Personalized PageRank is the main retrieval model.

Algorithm	P@10	HR@10	MRR
Common Neighbors	0.229641	0.711373	0.493400
Jaccard	0.219742	0.698766	0.420111
Adamic-Adar	0.230472	0.714056	0.502139
Personalized PR	**0.239297**	**0.791845**	**0.551049**

Table 2: Horizontal evaluation summary at $K = 10$ over five seeds. Values are mean Precision@10 and MRR.

Protocol	Best model	P@10	MRR
Random	Personalized PR	**0.237843**	**0.541522**
Degree matched	Jaccard/Sorensen	**0.141003**	**0.336657**
Hard 2-hop	Personalized PR	**0.222983**	**0.485723**

The implementation is intentionally modular: it evaluates the retrieval component of a GraphRAG pipeline before connecting it to a full LLM answering system. This separation makes retrieval behavior measurable without confounding it with generator quality, prompt design, or answer evaluation. In the current configuration, the most expensive component is approximate Personalized PageRank random-walk scoring over the directed training graph.

5 Results: Main-Model Performance and Horizontal Evaluation

Table 1 reports the saved full-dataset retrieval results for the original four-model experiment. Personalized PageRank, the paper's main retrieval model, is the strongest scorer on all three metrics. Compared with Adamic-Adar, the best local-overlap baseline in the original run, Personalized PageRank improves Precision@10 by 0.008825, HitRate@10 by 0.077789, and MRR by 0.048910. The largest gain appears in HitRate@10, indicating that source-conditioned random walks are particularly effective at bringing at least one hidden positive edge into the top retrieved context set.

The consolidated diagnostic figure provides the mechanism behind this result. Figure 2a confirms the aggregate advantage of Personalized PageRank in the original setting, while Figure 2b–c show that the model remains competitive across retrieval cutoffs rather than only at $K = 10$. This matters for GraphRAG because a downstream generator may consume different context budgets depending on prompt length, model context window, and latency constraints. A scorer that remains stable across cutoffs is easier to deploy than one that only performs well at a narrowly tuned K.

Figure 2d shows the score-level separation between positive hidden edges and sampled negatives. Successful retrieval requires not only high scores for positives, but also sufficient separation from plausible distractors. Personalized PageRank benefits from multi-hop directed evidence, whereas local overlap methods can assign similar low scores when two candidate nodes share few immediate neighbors. Figure 2e further shows that the training graph has a heavy-tailed degree distribution, which is typical of social and voting networks. Figure 2f illustrates the practical output of the main model: the retrieval module produces a ranked context set with structural evidence such as shared neighbors or short graph paths.

The horizontal evaluation broadens the conclusion from a single saved run to a multi-seed comparison. Table 2 and Figure 3a show that Personalized PageRank remains the best algorithm under the random-negative setting, with mean Precision@10 of 0.237843 and mean MRR of 0.541522 over five seeds. The same model also leads under hard two-hop negatives, where it reaches mean Precision@10 of 0.222983 and mean MRR of 0.485723. This is the strongest evidence for using Personalized PageRank as the main GraphRAG retriever: it performs well not only against easy random non-edges, but also against structurally nearby false candidates.

The degree-matched protocol reveals a different behavior. When negative targets are sampled to resemble positives in target in-degree, normalized local-overlap methods become stronger: Jaccard and Sorensen tie for the best mean Precision@10 at 0.141003 and mean MRR at 0.336657. This result is informative rather than contradictory. Personalized PageRank exploits directed multi-hop relevance and target reachability; degree matching removes part of the popularity signal and rewards algorithms that normalize local neighborhood overlap. The horizontal evaluation therefore shows a clear trade-off: Personalized PageRank is the best main model for the repository's default and hard-structural retrieval settings, while normalized local heuristics provide useful robustness baselines when target popularity is controlled.

Figure 3b extends the comparison across K. The ranking curves indicate that no single local baseline dominates across all retrieval budgets, while Personalized PageRank maintains strong top-rank behavior under the primary candidate protocols. Figure 4a further shows that negative sampling is not a minor implementation detail; it changes the measured difficulty of the retrieval task and therefore must be reported alongside algorithm metrics. Figure 4b places quality against runtime. Target In-Degree and Preferential Attachment are fast but weaker, local overlap methods provide a favorable cost-quality balance, and Personalized PageRank delivers the strongest retrieval quality at a higher computational cost.

Overall, the results support three observations. First, graph-only retrieval is feasible on the full Wiki-Vote network without neural training. Second, local graph heuristics provide strong and interpretable baselines, but their effectiveness depends heavily on the candidate protocol. Third, approximate Personalized PageRank is the best main model for this repository because it preserves directionality, uses multi-hop evidence, and produces the strongest ranking metrics under both the original random-negative evaluation and the harder two-hop evaluation.

6 Conclusion

This project demonstrates a reproducible GraphRAG retrieval module by turning context retrieval into link prediction on the Wiki-Vote graph. The main retrieval model is approximate Personalized PageRank, and the experiments show that it provides the strongest overall retrieval quality among the implemented scorers. In the original full-dataset setting, Personalized PageRank outperforms Common Neighbors, Jaccard, and Adamic-Adar on Precision@10, HitRate@10, and MRR. In the horizontal evaluation, it remains the best model under random negative sampling and hard two-hop negative sampling across five seeds.

The broader result is that graph-aware retrieval is valuable when relevant context is relational rather than purely textual. Personalized PageRank is especially well aligned with this setting because it uses directed multi-hop evidence while remaining inspectable and reproducible. The horizontal evaluation also clarifies the role of candidate construction: degree-matched negatives reduce the advantage of random-walk relevance and highlight normalized local-overlap methods, whereas hard two-hop negatives preserve the advantage of the main Personalized PageRank model. This gives the system a clearer empirical profile than a single baseline experiment.

Future work should connect the retrieved graph context to an LLM generation stage, add textual node attributes, test graph-plus-vector hybrid retrieval, and measure the downstream effect of graph retrieval on answer factuality and citation quality.

References

[1] Lada A. Adamic and Eytan Adar. Friends and neighbors on the web. *Social Networks*, 25(3):211–230, 2003. doi: 10.1016/S0378-8733(03)00009-1. URL `https://doi.org/10.1016/S0378-8733(03)00009-1`.

[2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2024. URL `https://arxiv.org/abs/2404.16130`.

[3] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 855–864, New York, NY, USA, 2016. ACM. doi: 10.1145/2939672.2939754. URL `https://doi.org/10.1145/2939672.2939754`.

[4] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008. doi: 10.25080/TCWV9851. URL `https://proceedings.scipy.org/articles/TCWV9851`.

[5] Glen Jeh and Jennifer Widom. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web*, WWW '03, pages 271–279, New York, NY, USA, 2003. ACM. doi: 10.1145/775152.775191. URL `https://doi.org/10.1145/775152.775191`.

[6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL `https://aclanthology.org/2020.emnlp-main.550`.

[7] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. `https://snap.stanford.edu/data`, June 2014. URL `https://snap.stanford.edu/data/wiki-Vote.html`.

[8] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Predicting positive and negative links in online social networks. In *Proceedings of the 19th International Conference on World Wide Web*, WWW '10, pages 641–650, New York, NY, USA, 2010. ACM. doi: 10.1145/1772690.1772756. URL `https://doi.org/10.1145/1772690.1772756`.

[9] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Signed networks in social media. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '10, pages 1361–1370, New York, NY, USA, 2010. ACM. doi: 10.1145/1753326.1753532. URL `https://doi.org/10.1145/1753326.1753532`.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL `https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html`.

[11] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007. doi: 10.1002/asi.20591. URL `https://doi.org/10.1002/asi.20591`.

[12] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report SIDL-WP-1999-0120, Stanford InfoLab, 1999. URL `https://ilpubs.stanford.edu/422/`.

[13] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '14, pages 701–710, New York, NY, USA, 2014. ACM. doi: 10.1145/2623330.2623732. URL `https://doi.org/10.1145/2623330.2623732`.

[14] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL `https://doi.org/10.1561/1500000019`.